

Práctica Github

Asignatura

Administración de Bases De Datos

Nombre del profesor/a

Dr. Richard Avilés López

Máster o Grado

Ingeniería Informática

Realizado por:

Xavier Alejandro Gallo Rodas



Índice de contenidos

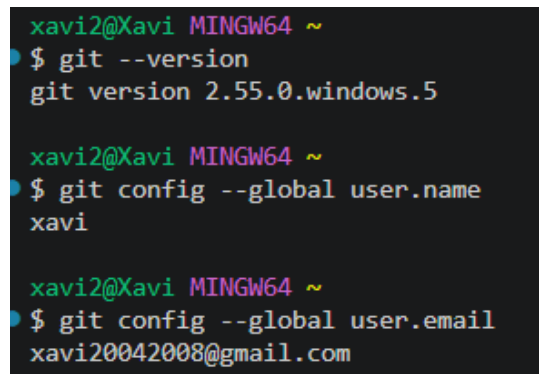
1. Introducción y objetivos:	3
2. Primeros pasos en github:	3
Parte A crear proyecto	4
Parte B: inicialización	4
Parte C: ARBOL Y ESTRUCTURA DE DATOS	5
Parte D: EDITAR Y COMMIT	6
PARTE E: EQUIVOCACION Y SOLUCIÓN	9
PARTE F: LEER HISTORIAL	11
PARTE G: BRANCHES crear, cambiar, comprobar que NO es una carpeta	11
PARTE H PROVOCAR Y RESOLVER CONFLICTOS	13
PARTE I CONECTAR CON GITHUB	16
3. PREGUNTAS DE LA PRÁCTICA:	18

1. Introducción y objetivos:

Al finalizar este laboratorio, el estudiante será capaz de:

- Verificar y configurar Git correctamente en su máquina (identidad de autor).
- Crear un repositorio desde cero y explicar qué hace exactamente `git init`.
- Diseñar una estructura de carpetas profesional para un proyecto real.
- Interpretar correctamente la salida de `git status` en cualquier situación.
- Utilizar la Staging Area de forma consciente (`git add`, `git diff`, `git diff --staged`).
- Crear commits atómicos y bien redactados siguiendo Conventional Commits.
- Cometer un error típico de principiante, diagnosticarlo y corregirlo con seguridad.
- Leer e interpretar el historial (`git log` en sus variantes más útiles).
- Crear, cambiar y eliminar branches, entendiendo que una branch NO es una carpeta.
- Provocar un conflicto de fusión de forma controlada y resolverlo sin pánico.
- Conectar un repositorio local con GitHub y realizar `push` y `pull` correctamente.
- Explicar con sus propias palabras la diferencia entre Git y GitHub.

2. Primeros pasos en github:



```
xavi2@Xavi MINGW64 ~  
$ git --version  
git version 2.55.0.windows.5  
  
xavi2@Xavi MINGW64 ~  
$ git config --global user.name  
xavi  
  
xavi2@Xavi MINGW64 ~  
$ git config --global user.email  
xavi20042008@gmail.com
```

Se comprueba versión y usuario.

Parte A crear proyecto

```
xavi2@Xavi MINGW64 ~
• $ cd oracle-database-lab

xavi2@Xavi MINGW64 ~/oracle-database-lab
• $ ls

xavi2@Xavi MINGW64 ~/oracle-database-lab
• $ ls -la
total 24
drwxr-xr-x 1 xavi2 197609 0 Sep 18 17:56 ./
drwxr-xr-x 1 xavi2 197609 0 Sep 18 17:56 ../
```

Ls -la no muestra carpetas.git

Parte B: inicialización

```
xavi2@Xavi MINGW64 ~/oracle-database-lab
• $ git init
Initialized empty Git repository in C:/Users/xavi2/oracle-database-lab/.git/

xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
• $ ls -la
total 28
drwxr-xr-x 1 xavi2 197609 0 Sep 18 18:08 ./
drwxr-xr-x 1 xavi2 197609 0 Sep 18 17:56 ../
drwxr-xr-x 1 xavi2 197609 0 Sep 18 18:08 .git/
```

Ahora si aparece un .git Esa carpeta es la infraestructura interna de Git: aquí vivirán todos los commits, branches y configuración del repositorio.

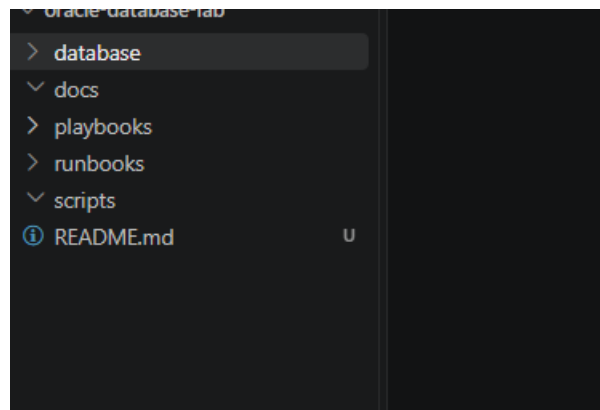
```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
• $ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

OJO: Nunca edites archivos dentro de .git a mano. Es la base de datos interna de Git. Toda interacción debe hacerse mediante comandos `git ...`. Ejecutar `git init` dentro de una carpeta que ya está dentro de otro repositorio Git crea un "repositorio anidado" que confunde a Git sobre qué .git debe usar. Antes de hacer `git init`, comprueba con `git status` si ya estás dentro de un repositorio existente

Parte C: ARBOL Y ESTRUCTURA DE DATOS



Se han creado con los comandos: `mkdir -p database/schema database/migrations database/rollback database/tests`

`mkdir -p scripts runbooks playbooks docs`

`touch README.md`

`git status`

OJO: Git no versiona carpetas, versiona archivos. Una carpeta completamente vacía no deja ningún rastro que Git pueda registrar. Solución profesional: si necesitas conservar una carpeta vacía en el repositorio (por ejemplo, `runbooks/` antes de escribir el primer runbook), coloca dentro un archivo placeholder, normalmente llamado `.gitkeep` (convención, no es una palabra reservada de Git) o un `README.md` explicando el propósito de la carpeta.

Comandos para esto:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ touch runbooks/.gitkeep playbooks/.gitkeep database/schema/.gitkeep database/migrations/.gitkeep database/rollback/.gitkeep database/tests/.gitkeep scripts/.gitkeep

xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    database/
    playbooks/
    runbooks/
    scripts/

nothing added to commit but untracked files present (use "git add" to track)
```

Para ver la estructura del árbol:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ cmd //c tree
find . -print | sed -e 's;[^\/*];| ;g'
Listado de rutas de carpetas para el volumen OS
El número de serie del volumen es 9058-C62B
C:..
├── database
│   ├── migrations
│   ├── rollback
│   ├── schema
│   └── tests
├── docs
├── playbooks
├── runbooks
└── scripts

.
├── .git
│   ├── config
│   ├── description
│   ├── HEAD
│   ├── hooks
│   │   ├── applypatch-msg.sample
│   │   ├── commit-msg.sample
│   │   ├── fsmonitor-watchman.sample
│   │   ├── post-update.sample
│   │   ├── pre-applypatch.sample
│   │   ├── pre-commit.sample
│   │   ├── pre-merge-commit.sample
│   │   └── pre-push.sample
```

Parte D: EDITAR Y COMMIT

1. Abrir read me editarlo y luego vamos a observar el estado:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git config --global status.showUntrackedFiles all

xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    README.md
    database/migrations/.gitkeep
    database/rollback/.gitkeep
    database/schema/.gitkeep
    database/tests/.gitkeep
    playbooks/.gitkeep
    runbooks/.gitkeep
    scripts/.gitkeep

nothing added to commit but untracked files present (use "git add" to track)
```

Se ven en rojo los “Untracked Files” significa que Git ve el archivo en el disco, pero todavía no lo sigue.

2. Ahora vamos a moverla a Staging Area

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git add README.md

xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    database/migrations/.gitkeep
    database/rollback/.gitkeep
    database/schema/.gitkeep
    database/tests/.gitkeep
    playbooks/.gitkeep
    runbooks/.gitkeep
    scripts/.gitkeep

xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$
```

Ahora README.md aparece en verde, bajo "Changes to be committed". Ha pasado del Working Directory a la Staging Area. Todavía no existe ningún commit.

3. Ahora como buena práctica vamos a revisar antes de hacer el commit

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git diff --staged
diff --git a/README.md b/README.md
new file mode 100644
index 0000000..a088ccb
--- /dev/null
+++ b/README.md
@@ -0,0 +1,5 @@
+# Oracle Database Lab
+Training repository for Oracle Database administration,
+testing, change management and Git workflows.
+Name: XAVIER ALEJANDRO GALLO RODAS
+Professor: Richard Aviles Lopez
+ \ No newline at end of file
```

4. Primer commit con conventional commit

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git commit -m "docs: add initial project documentation"
git status
git log
[master (root-commit) d272422] docs: add initial project docum
entation
 1 file changed, 5 insertions(+)
 create mode 100644 README.md
On branch master
Untracked files:
 (use "git add <file>..." to include in what will be committe
d)

 database/migrations/.gitkeep
 database/rollback/.gitkeep
 database/schema/.gitkeep
 database/tests/.gitkeep
 playbooks/.gitkeep
 runbooks/.gitkeep
 scripts/.gitkeep

nothing added to commit but untracked files present (use "git
add" to track)
commit d272422807c1341162fd01755f890089da3c5174 (HEAD -> maste
r)
Author: xavi <xavi20042008@gmail.com>
Date:   Fri Sep 18 18:35:48 2026 +0200

 docs: add initial project documentation
```

Se ve como `git log` muestra el historial de commits en el repositorio, en este caso, mi primer commit, con su hash, autor, fecha y mensaje.

El árbol no está limpio, no todos los archivos están con commit

OJO: El código hash sirve para identificar un commit sin ambigüedad, volver a un punto exacto del historial y comparar versiones.

Conventional Commits — el estandar que usaremos todo el curso

La especificación Conventional Commits define la estructura:

Formato

```
<tipo>[ámbito opcional]: <descripción>
```

Tipo	Cuándo usarlo
feat	Añade una funcionalidad nueva
fix	Corrige un error
docs	Cambios solo en documentación
test	Añade o corrige tests
refactor	Cambia el código sin alterar el comportamiento
perf	Mejora de rendimiento
build / ci	Cambios en compilación o en la pipeline de integración continua
chore	Tareas de mantenimiento (dependencias, configuración menor)
security	Cambios relacionados con seguridad

OJO: con los mensajes de commit, que no sean ambiguos, además es de buena práctica Un commit debe representar un único cambio lógico. No mezcles "crear tabla + arreglar tests + actualizar documentación" en un solo commit: divide en varios commits. Esto facilita enormemente la revisión, el diagnóstico y el rollback futuro.

PARTE E: EQUIVOCACION Y SOLUCIÓN

Ejercicio 1: commit a medias

Crea un nuevo archivo de documentación, pero olvida añadirlo antes de confirmar:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ echo "# Database schema notes" > docs/customer-schema.md
git commit -m "docs: add customer schema notes"
git status
git log --oneline
```

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ echo "# Database schema notes" > docs/customer-schema.md
git commit -m "docs: add customer schema notes"
git status
git log --oneline
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        database/migrations/.gitkeep
        database/rollback/.gitkeep
        database/schema/.gitkeep
        database/tests/.gitkeep
        docs/customer-schema.md
        playbooks/.gitkeep
        runbooks/.gitkeep
        scripts/.gitkeep

nothing added to commit but untracked files present (use "git add" to track)
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        database/migrations/.gitkeep
        database/rollback/.gitkeep
        database/schema/.gitkeep
        database/tests/.gitkeep
        docs/customer-schema.md
        playbooks/.gitkeep
        runbooks/.gitkeep
        scripts/.gitkeep

nothing added to commit but untracked files present (use "git add" to track)
```

Git responde algo como "nothing added to commit but untracked files present" — el commit NO se realizó, porque el archivo nunca llegó a la Staging Area. git commit únicamente confirma lo que está en la Staging Area. Si el archivo es nuevo (untracked) y no hiciste `git add`, el commit simplemente no lo incluye.

Diagnóstico:

- Ejecuta siempre `git status` inmediatamente antes de confirmar, y
- `git diff --staged` para ver exactamente qué vas a grabar.

Corrección:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git add docs/customer-schema.md
git commit -m "docs: add customer schema notes"
git log --oneline
warning: in the working copy of 'docs/customer-schema.md', LF
will be replaced by CRLF the next time Git touches it
[master 3d0f2ab] docs: add customer schema notes
1 file changed, 1 insertion(+)
create mode 100644 docs/customer-schema.md
3d0f2ab (HEAD -> master) docs: add customer schema notes
d272422 docs: add initial project documentation
```

¿COMO VER LOS COMMITS QUE HE REALIZADO?

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git log --oneline
3d0f2ab (HEAD -> master) docs: add customer schema notes
d272422 docs: add initial project documentation
```

EJERCIO 2: EL MENSAJE DEL COMMIT CON ERROR

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ echo "Some content" >> docs/customer-schema.md
git add docs/customer-schema.md
git commit -m "dcos: fxi typo"
warning: in the working copy of 'docs/customer-schema.md', LF
will be replaced by CRLF the next time Git touches it
[master c06583c] dcos: fxi typo
1 file changed, 1 insertion(+)

xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$
```

Se puede corregir si el commit no se ha subido a github push , se puede corregir el ultimo con -amend

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git commit --amend -m "docs: fix typo in schema notes"
git log --oneline
[master 9aa0776] docs: fix typo in schema notes
Date: Fri Sep 18 18:50:34 2026 +0200
1 file changed, 1 insertion(+)
9aa0776 (HEAD -> master) docs: fix typo in schema notes
3d0f2ab docs: add customer schema notes
d272422 docs: add initial project documentation
```

Esta solución reescribe el historial, por tanto -amend es seguro usarlo siempre y cuando el commit sea solo local. En cuanto hagas `git push`, considera ese commit "público" e inmutable; para corregirlo, haz un commit nuevo.

PARTE F: LEER HISTORIAL

Comandos de consulta más usados:

```
git log                # historial completo y detallado
git log --oneline      # una línea por commit
git log --oneline --graph --all  # con gráfico de branches
git show <hash>        # detalle de un commit concreto
git diff               # cambios sin stage aún
git diff --staged      # cambios ya en la Staging Area
```

OJO: EL MAS IMPORTANTE ES `git log --oneline --graph --all`

PARTE G: BRANCHES crear, cambiar, comprobar que NO es una carpeta

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git branch
* master

xavi2@Xavi MINGW64 ~/oracle-database-lab (master)
$ git switch -c feature/customer-search
git branch
ls -la
Switched to a new branch 'feature/customer-search'
* feature/customer-search
  master
total 33
drwxr-xr-x 1 xavi2 197609  0 Sep 18 18:12 ./
drwxr-xr-x 1 xavi2 197609  0 Sep 18 18:57 ../
drwxr-xr-x 1 xavi2 197609  0 Sep 18 18:57 .git/
drwxr-xr-x 1 xavi2 197609  0 Sep 18 18:12 database/
drwxr-xr-x 1 xavi2 197609  0 Sep 18 18:43 docs/
drwxr-xr-x 1 xavi2 197609  0 Sep 18 18:19 playbooks/
-rw-r--r-- 1 xavi2 197609 194 Sep 18 18:29 README.md
drwxr-xr-x 1 xavi2 197609  0 Sep 18 18:19 runbooks/
drwxr-xr-x 1 xavi2 197609  0 Sep 18 18:25 scripts/
```

Ojo: ls muestra las mismas carpetas y archivos que antes porque la Branch es un puntero dentro de .git no una carpeta del disco

EJEMPLO DE COMMIT DENTRO DE ESTRA BRANCH

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (feature/customer-search)
$ echo "Draft: customer search notes" > docs/customer-search.md
git add docs/customer-search.md
git commit -m "docs: draft customer search notes"
git log --oneline --graph --all
warning: in the working copy of 'docs/customer-search.md', LF
will be replaced by CRLF the next time Git touches it
[feature/customer-search 2d66286] docs: draft customer search
notes
1 file changed, 1 insertion(+)
create mode 100644 docs/customer-search.md
* 2d66286 (HEAD -> feature/customer-search) docs: draft custom
er search notes
* 9aa0776 (master) docs: fix typo in schema notes
* 3d0f2ab docs: add customer schema notes
* d272422 docs: add initial project documentation
```

¿A dónde apunta el HEAD? ¿En qué orden aparecen los commits? Responde:

EL head apunta a feature/customer-search, los commits aparecen en orden cronológico inverso (del más reciente al más antiguo)

COMO CAMBIAR DE NOMBRE AL MAIN Y VER COMO EL CONTENIDO DEL DISCO CAMBIA AL CAMBIAR LA BRANCH

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (feature/customer-search)
$ git switch main
ls docs/
git switch feature/customer-search
ls docs/
fatal: invalid reference: main
customer-schema.md customer-search.md
Already on 'feature/customer-search'
customer-schema.md customer-search.md

xavi2@Xavi MINGW64 ~/oracle-database-lab (feature/customer-search)
$ git branch -m master main

xavi2@Xavi MINGW64 ~/oracle-database-lab (feature/customer-search)
$ git config --global init.defaultBranch main

xavi2@Xavi MINGW64 ~/oracle-database-lab (feature/customer-search)
$ git switch main
ls docs/
git switch feature/customer-search
ls docs/
Switched to branch 'main'
customer-schema.md
Switched to branch 'feature/customer-search'
customer-schema.md customer-search.md
```

Al volver a main, customer-search.md desaparece del disco (porque ese commit no existe en main); al volver a la branch, reaparece. Git está reescribiendo el contenido del Working Directory según a qué commit apunta la branch activa —sin haber creado ninguna carpeta nueva.

OJO git checkout -b nombre-branch git switch -c nombre-branch es lo mismo

Como eliminar una branch local (solo si ya esta fusionada)

git branch -d nombre-de-una-branch-de-prueba

⚠ ERROR FRECUENTE — "Empecé a trabajar directamente en main sin querer"

Es el error más común de la primera semana: abrir el editor y modificar archivos estando en main, sin haber creado antes una branch.

Si aún no has hecho commit, puedes crear la branch ahora mismo con los cambios sin confirmar

Git los traslada contigo: `git switch -c feature/nombre` (los cambios sin commit no se pierden, viajan a la branch nueva).

PARTE H PROVOCAR Y RESOLVER CONFLICTOS

Imaginando que hay dos desarrolladores:

1. Volver al main y crear un Branch A

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (feature/customer-search)
$ git switch main
git switch -c fix/readme-title
Switched to branch 'main'
Switched to a new branch 'fix/readme-title'
```

EDITO LA PRIMERA LINEA DEL README

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (fix/readme-title)
$ git status
git add README.md
git status
git commit -m "docs: rename project title (training edition)"
git status -c
```

¿QUE PASO CON GIT STATUS?

El comando arrojó el error error: unknown switch 'c' porque se ejecutó con una opción no reconocida, al no reconocer ese modificador, Git imprimió el menú de ayuda del comando.

Y en git status me muestra que readme ha sido modificado, pero no está confirmado aún.

2. Crear una Branch b en main y se modifica la misma línea

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (fix/readme-title)
$ git switch main
git switch -c fix/readme-subtitle
Switched to branch 'main'
Switched to a new branch 'fix/readme-subtitle'

xavi2@Xavi MINGW64 ~/oracle-database-lab (fix/readme-subtitle)
$ git add README.md
git commit -m "docs: rename project title (academic version)"
git switch main
[fix/readme-subtitle b4607d2] docs: rename project title (academic version)
1 file changed, 1 insertion(+), 1 deletion(-)
Switched to branch 'main'
```

3. Fusión la primera Branch

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (main)
$ git merge fix/readme-title
Updating 9aa0776..6f58dd1
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

sin error

4. Fusión la siguiente

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (main)
$ git merge fix/readme-subtitle
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

tengo conflicto

```
1 <<<<<< HEAD (Current Change)
2 # Oracle Database Lab (Training Edition)
3 =====
4 # Oracle Database Lab - Academic Version
5 >>>>>> fix/readme-subtitle (Incoming Change)
6 Training repository for Oracle Database administration,
7 testing, change management and Git workflow.
  Resolve in Merge Editor
```

Marcador	Significado
<<<<<< HEAD	Empieza la versión que ya tenías en tu branch actual (main)
=====	Separador entre las dos versiones en conflicto
>>>>>> fix/readme-subtitle	Termina la versión que venía de la branch que estás fusionando

5. Resolver

Decide (o combina) el contenido final, elimina TODOS los marcadores y deja el archivo limpio, por ejemplo:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (main|MERGING)
$ git status
git add README.md
git commit -m "merge: resolve README title conflict"
git log --oneline --graph --all
On branch main
All conflicts fixed but you are still merging.
(use "git commit" to conclude merge)

Changes to be committed:
  modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    database/migrations/.gitkeep
    database/rollback/.gitkeep
    database/schema/.gitkeep
    database/tests/.gitkeep
    playbooks/.gitkeep
    runbooks/.gitkeep
    scripts/.gitkeep

[main c6f5f59] merge: resolve README title conflict
* c6f5f59 (HEAD -> main) merge: resolve README title conflict
| \
| * b4607d2 (fix/readme-subtitle) docs: rename project title (academic version)
| * 6f58dd1 (fix/readme-title) docs: rename project title (training edition)
| /
```

⚠ ERROR FRECUENTE — Dejar los marcadores <<<<<< sin quitar

El error más frecuente al resolver un conflicto es hacer `git add` y `git commit` sin haber borrado realmente los marcadores `<<<<<<`, `====`, `>>>>>>`. El commit se completa "sin errores" pero el archivo queda roto (código o documentación inválidos). Después de resolver, siempre revisa el archivo entero antes de confirmar, y si es código, ejecútalo o testéalo.

✓ BUENA PRÁCTICA — "Borro todo y empiezo de cero" (pánico ante un conflicto)

Un conflicto NO es un error grave ni una señal de que algo se ha roto: es Git pidiéndote una decisión que solo un humano puede tomar.

Si te bloqueas, te pones nervioso o te da ansiedad, siempre puedes abortar el merge sin perder nada y volver al estado anterior con `git merge --abort`, y reintentarlo con calma.

Si quieres empezar de nuevo esta parte borra las ramas desde main con `git branch -d fix/readme-title`
`git Branch -d fix/readme-subtitle`

PARTE I CONECTAR CON GITHUB

Paso 1 — Crear el repositorio remoto (en la web de GitHub)

1. Entra en github.com con tu cuenta.
2. Pulsa "New repository".
3. Nombre: oracle-database-lab.
4. NO marques "Add a README" (ya tenemos uno local) para evitar historiales que no encajan.
5. Crea el repositorio y copia la URL que te ofrece (HTTPS).

Paso 2 — Vincular tu repositorio local con el remoto

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (main)
$ git remote add origin https://github.com/xagallo-design/oracle-database-lab.git

xavi2@Xavi MINGW64 ~/oracle-database-lab (main)
$ git remote -v
git branch -M main
git push -u origin main
origin https://github.com/xagallo-design/oracle-database-lab.git
git (fetch)
origin https://github.com/xagallo-design/oracle-database-lab.git
git (push)
info: please complete authentication in your browser...
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 16 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (20/20), 1.84 KiB | 628.00 KiB/s, done.
Total 20 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/xagallo-design/oracle-database-lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Paso 3 — Subir el resto de branches (opcional para este laboratorio)

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (main)
$ git push -u origin fix/readme-title
git push -u origin fix/readme-subtitle
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'fix/readme-title' on GitHub
remote: by visiting:
remote:   https://github.com/xagallo-design/oracle-database-lab/pull/new/fix/readme-title
remote:
To https://github.com/xagallo-design/oracle-database-lab.git
 * [new branch]      fix/readme-title -> fix/readme-title
branch 'fix/readme-title' set up to track 'origin/fix/readme-title'.
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'fix/readme-subtitle' on GitHub
remote: by visiting:
remote:   https://github.com/xagallo-design/oracle-database-lab/pull/new/fix/readme-subtitle
remote:
To https://github.com/xagallo-design/oracle-database-lab.git
 * [new branch]      fix/readme-subtitle -> fix/readme-subtitle
branch 'fix/readme-subtitle' set up to track 'origin/fix/readme-subtitle'.
```

Paso 4 — Simular un cambio hecho "desde otro sitio" y traerlo (pull)

Edita un archivo directamente en la interfaz web de GitHub (por ejemplo, añade una línea a README.md desde el navegador) en la rama fix/readme-subtitle y confírmalo ahí. Después, en tu terminal:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (main)
$ cat README.md
git switch fix/readme-subtitle
git pull
cat README.md
# Oracle Database Lab (Training Edition – Academic Version)
Training repository for Oracle Database administration,
testing, change management and Git workflows.
Name: XAVIER ALEJANDRO GALLO RODAS
Professor: Richard Aviles LopezSwitched to branch 'fix/readme-
subtitle'
Your branch is up to date with 'origin/fix/readme-subtitle'.
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (
from 0)
Unpacking objects: 100% (3/3), 1.00 KiB | 128.00 KiB/s, done.
From https://github.com/xagallo-design/oracle-database-lab
c6f5f59..d052dfd main -> origin/main
Already up to date.
# Oracle Database Lab – Academic Version
Training repository for Oracle Database administration,
testing, change management and Git workflows.
Name: XAVIER ALEJANDRO GALLO RODAS
Professor: Richard Aviles Lopez
```

Git pull trae de github solo lo que corresponda a la rama en la que estas parado. No trae todo el main y todas las ramas

3. PREGUNTAS DE LA PRÁCTICA:

1. Diferencia entre Working Directory, Staging Area y Local Repository

- **Working Directory:** Tu carpeta con los archivos que estás editando.
- **Staging Area:** La "bandeja" donde pones los cambios que quieres incluir en la foto.
- **Local Repository:** El álbum permanente (.git) donde se guardan las fotos confirmadas (commits).
- *Ejemplo:* Editas README.md (Working Directory), haces git add README.md (Staging Area) y haces git commit (Local Repository).

2. Si modificas un archivo y no haces git add, ¿entra en el próximo commit?

No. git commit solo guarda lo que hayas subido antes a la Staging Area con git add.

3. ¿Por qué git status no mostraba las carpetas vacías y qué truco se usó? Git solo rastrea archivos, no carpetas vacías. Se solucionó creando un archivo vacío dentro llamado .gitkeep para obligar a Git a registrar la carpeta.

4. ¿Qué es HEAD? Es el marcador o puntero que dice: "estás trabajando exactamente aquí" (en esta rama y en este commit).

5. Diferencia entre rama (git switch -c) y carpeta (mkdir), y comprobación mkdir crea una carpeta física en tu disco. Una rama es solo una línea de tiempo paralela que no crea carpetas. Se comprobó al ver con ls -la que no existía ninguna carpeta nueva y que los archivos en disco cambiaban solos al saltar de una rama a otra con git switch.

6. Marcadores de conflicto en la Parte H

- Entre <<<<<< HEAD y =====: El texto que ya tenías en la rama donde estás parado (main).
- Entre ===== y >>>>>>: El texto que viene de la rama que intentas fusionar (fix/readme-subtitle).

7. ¿Por qué NO hacer git commit --amend si ya hiciste push? Porque --amend destruye el commit viejo y crea uno nuevo con otro identificador. Al estar ya subido en GitHub, romperás el historial de los demás compañeros de equipo.

8. Si borras la carpeta .git, ¿qué se pierde y qué pasa con el código? Se pierde todo el historial, las ramas y la configuración de Git. Tus archivos y tu código actual en el disco no se pierden.

9. Diferencia entre Git y GitHub (sin decir "nube")

- **Git:** El programa que corre en tu ordenador para registrar tus cambios de código.

- **GitHub:** La página web externa donde guardas una copia de tu proyecto para compartirlo y trabajar con otros.

10. ¿Por qué no subir un .env con claves reales aunque el repo sea privado?

Porque queda grabado para siempre en el historial de commits y cualquier persona con acceso al proyecto (o si se hace público por error) podrá ver tus contraseñas.

11. ¿Qué significa el error "non-fast-forward" y qué ejecutas primero? Significa que el repositorio remoto tiene cambios que tú no tienes en tu máquina. Primero ejecutas:

- Bash
- git pull
- Para descargar e integrar esos cambios antes de volver a intentar el push.

12. Tipos de Conventional Commits para cada caso

- **Añadir índice de rendimiento:** perf
- **Corregir restricción mal definida:** fix
- **Actualizar el README:** docs